Guías de los notebooks

Una guía por cada notebook del curso, escrita para alguien que empieza desde cero. Cada guía tiene:

- qué vas a aprender;
- cómo abrir el notebook;
- la teoría mínima, explicada con palabras simples;
- un recorrido **celda por celda**, con pistas para cada `TODO` y **valores para comprobar** tu resultado (salen de ejecutar el notebook de verdad);
- respuestas orientativas a las preguntas del notebook;
- una tabla de errores frecuentes.

Las guías **no** dan el código de la solución: dan pistas y resultados para comprobar.

#	Notebook	Tema	Clase	Puntos	Guía
1	`1_01_mcculloch_pitts`	Neurona de McCulloch y Pitts	2	3	Guía
2	`1_02_perceptron_AND`	Perceptrón: pesos a mano para AND	2	práctica	Guía
3	`1_03_1_perceptron_simple`	Perceptrón simple en Python	2	3	Guía
4	`1_03_2_regla_de_aprendizaje`	Regla de aprendizaje de Rosenblatt	2	4	Guía
5	`1_04_red_neuronal_simple`	Neurona con sigmoide y tanh	3	3	Guía
6	`1_05_descenso_por_gradiente_1D`	Descenso por gradiente en una variable	3	2	Guía
7	`1_06_descenso_unpaso`	Un paso de la regla delta	4	3	Guía
8	`1_07_descenso_por_gradiente`	Entrenar una neurona completa	4	3	Guía
9	`1_08_adaline`	ADALINE: regresión con una neurona	4	demostración	Guía
10	`1_09_red_multicapa_ejercicio`	Red multicapa: pase hacia adelante	5	3	Guía
11	`1_10_retropropagacion_base`	Retropropagación: un paso	5	3	Guía
12	`1_11_retropropagacion_completo`	Entrenar una red multicapa	5	3	Guía
13	`21_opt_gradiente_2d`	Descenso por gradiente en 2D	6	experimentos	Guía
14	`22_opt_momentum`	Optimizador con momentum	6	experimentos	Guía
15	`23_opt_lr_decay`	Decaimiento de la tasa de aprendizaje	6	3	Guía
16	`3_1_capa_convolucional`	Capa convolucional (correlación cruzada)	7	4	Guía

#	Notebook	Tema	Clase	Puntos	Guía
17	`3_2_entrenar_capa_conv`	Reto: entrenar un kernel convolucional	7	reto	Guía

La columna **Clase** indica en qué clase se trabaja cada notebook; cada clase tiene su guía (`docs/GUIA_PASO_A_PASO_CLASEn.md`) y sus diapositivas en `entregables/`.

Antes de cualquier notebook

Necesitas el ambiente `nni` y el repositorio descargado: Guía de la Clase 1, Parte A. Cada guía repite en corto cómo abrir su notebook.

Cambios respecto a los notebooks originales

- **`23_opt_lr_decay`:** en el repositorio original está **vacío**. Este curso incluye uno nuevo, en el mismo estilo que los notebooks 21 y 22.
- **`1_03_2_regla_de_aprendizaje`:** se agregó `n_classes=2` en las dos gráficas, porque scikit-learn 1.9 lo exige.
- **`notebooks/output/`:** carpeta nueva, donde `1_05` guarda su gráfica.

Guía del notebook `1_01_mcculloch_pitts`: la neurona de McCulloch y Pitts

Puntos: 3 · **Tiempo:** 20–30 min · **Clase:** 2 · **Archivo:** `notebooks/1_01_mcculloch_pitts.ipynb`
Teoría completa: Guía de la Clase 2, sección B2.

¿Qué vas a aprender?

- Cómo decide la primera neurona artificial de la historia (1943).
- A traducir unas reglas escritas en palabras a una función de Python con `for` e `if`.

Antes de empezar

1. Abre **Anaconda PowerShell Prompt** y escribe `conda activate nni`.
2. Abre VS Code → **File** → **Open Folder** → `curso-redes-neuronales` → carpeta `notebooks` → `1_01_mcculloch_pitts.ipynb`.
3. Arriba a la derecha, elige el kernel **nni**.
4. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**. Las partes que tú completas dicen `TODO`.

> **Sin instalar nada:** en Google Colab, **Archivo** → **Abrir cuaderno** → **GitHub** → `datahuba/curso-redes-neuronales` → este notebook.

La idea en 2 minutos

La neurona se escribe **C(E, I, u)**:

- **E**: lista de entradas **excitatorias** (cada una vale 0 o 1). Empujan a la neurona a encenderse.
- **I**: lista de entradas **inhibitorias** (0 o 1). Si alguna vale 1, la neurona se apaga.
- **u**: el **umbral**: cuántas excitatorias encendidas hacen falta.

Reglas, **en este orden**:

1. Si **alguna** inhibitoria vale 1 → la salida es **0**.
2. Si no, suma las excitatorias: si la suma es **mayor o igual que u** → la salida es **1**.
3. En otro caso → **0**.

Analogía: un portero de discoteca. Las inhibitorias son la "lista negra": si estás ahí, no entras. Si no, entras solo si traes al menos *u* invitaciones (excitatorias).

Paso a paso

Celda "importamos paquetes"

Solo carga NumPy. Ejecútala.

Celda `TODO (2 puntos)`: la función `neuronaMyP(E, I, u)`

Reemplaza `pass` por tu código. **No uses funciones de NumPy**: solo `for`, `if` y sumas.

Pistas:

1. Recorre la lista `I` con un `for`. Si encuentras un 1, haz `return 0` de inmediato.
2. Crea una variable `suma = 0` y súmale cada elemento de `E` con otro `for`.
3. Al final, compara: si `suma >= u` devuelve 1; si no, 0.

Celda "suponga"

Ya trae `E = [1]`, `I = [0]` y `u = 1`. Agrega una línea que imprima `neuronaMyP(E, I, u)`.
✅ **Comprobación:** debe imprimir **1**: no hay inhibición y la suma (1) alcanza el umbral (1).

Celda `TODO (1 punto)`: un programa para cualquier E, I y u

Escribe código que pruebe la neurona con varios casos. Por ejemplo, una función `evaluar(E, I, u)` que imprima la entrada y la salida, y llámala con distintos valores.

✅ **Comprobación** (todos deben coincidir):

Llamada	Qué representa	Salida
`neuronaMyP([], [0], 0)`	NOT con entrada 0	1
`neuronaMyP([], [1], 0)`	NOT con entrada 1	0
`neuronaMyP([1, 1], [], 2)`	AND de (1, 1)	1
`neuronaMyP([1, 0], [], 2)`	AND de (1, 0)	0
`neuronaMyP([0, 1], [], 1)`	OR de (0, 1)	1
`neuronaMyP([1, 1, 0], [], 2)`	"al menos 2 de 3"	1

Errores frecuentes

Síntoma	Causa	Solución
Imprime `None`	Falta un `return` en algún camino	Revisa que la función siempre devuelva 0 o 1
`neuronaMyP([], [0], 0)` da 0	Olvidaste que con E vacío la suma es 0, y 0 ≥ 0	Con `>=` sale 1; revisa que no uses `>`
Una inhibitoria activa no apaga la neurona	Revisas I después de sumar, o no haces `return`	Revisa I **primero** y sal con `return 0`
`NameError: neuronaMyP is not defined`	No ejecutaste la celda de la función	Ejecútala antes de las de prueba

Guía del notebook `1_02_perceptron_AND`: pesos a mano para AND

Puntos: práctica, sin puntos · **Tiempo:** 5–10 min · **Clase:** 2 · **Archivo:** `notebooks/1_02_perceptron_AND.ipynb`
Teoría completa: Guía de la Clase 2, sección B3.

¿Qué vas a aprender?

- Que un perceptrón es una **recta** que separa dos grupos de puntos.
- A elegir pesos y sesgo **a mano** para que el perceptrón calcule la compuerta AND.

Antes de empezar

1. `conda activate nni` en Anaconda PowerShell Prompt.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_02_perceptron_AND.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El perceptrón calcula:

1. **Combinación lineal:** h = peso1·x_1 + peso2·x_2 + sesgo
2. **Activación escalón:** salida = 1 si h ≥ 0, y 0 si h < 0.

AND vale 1 **solo** cuando las dos entradas son 1:

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Necesitas que h sea **negativa** en los tres primeros casos y **positiva o cero** en el último.

Paso a paso

Celda `TODO`: `weight1`, `weight2` y `bias`

Cambia los tres valores (que empiezan en 0).

Pistas para razonar:

- Con los dos pesos iguales y positivos, el caso (1, 1) da la suma más grande.

- El sesgo debe ser negativo: lo bastante para "apagar" los casos con un solo 1, pero no tanto como para apagar el caso (1, 1).
- Prueba pesos de 1 y busca el sesgo entre −1 y −2.

Celda "ESTO DEBE QUEDAR IGUAL"

No la modifiques. Prueba tus pesos con las 4 entradas y te dice cuántas acertaste.

✅ **Comprobación:** debe decir `Bien! Acertaste todas.` y la columna `Is Correct` debe decir `Yes` en las cuatro filas. Por ejemplo, una solución válida da combinaciones lineales −1.5, −0.5, −0.5 y 0.5.

> Hay **muchas** soluciones correctas: cualquier recta que deje solo el punto (1, 1) del lado positivo.

Para pensar

- ¿Qué cambiarías para que calcule **OR** (1 si al menos una entrada es 1)? → Solo el sesgo, menos negativo.
- ¿Y NAND (lo contrario de AND)? → Todos los signos al revés.

Errores frecuentes

Síntoma	Causa	Solución
`Obtuvieste 1 mal` en la fila (1, 1)	El sesgo es demasiado negativo	Súbelo un poco (hazlo menos negativo)
Fallan las filas (0, 1) y (1, 0)	El sesgo no es lo bastante negativo	Bájalo
`NameError: weight1`	No ejecutaste la celda de los pesos	Ejecútala antes que la de prueba

Guía del notebook `1_03_1_perceptron_simple`: el perceptrón en Python

Puntos: 3 · **Tiempo:** 15–20 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_1_perceptron_simple.ipynb`
Teoría completa: Guía de la Clase 2, sección B3.

¿Qué vas a aprender?

- A programar las dos piezas del perceptrón: la **combinación lineal** y la **función escalón**.
- A unirlas para hacer una predicción ("pase frontal").

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_1_perceptron_simple.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Combinación lineal:** $h = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$. Multiplica cada entrada por su peso, suma todo y suma el sesgo.
- **Escalón:** devuelve 1 si $h \geq 0$, y 0 si $h < 0$.
- **Perceptrón:** salida = escalón(combinación lineal).

Paso a paso

Celda "Cargamos paquetes"

Carga NumPy. Ejecútala.

Celda `TODO (1 punto)` : `combinacion_lineal(W, X, b)`

Sin usar `np.dot`. Pistas:

1. Empieza con `suma = 0.0`.
2. Recorre los pares (peso, entrada) a la vez con `for w, x in zip(W, X):` y acumula `w * x`.
3. Al final devuelve la suma más `b`.

Celda `TODO (1 punto)` : `escalon(h)`

Debe devolver 1 si $h \geq 0$ y 0 si no, y funcionar tanto con un número como con un arreglo de NumPy.

Pista: la comparación `h >= 0` da `True` / `False`. Conviértela a entero, por ejemplo con `np.asarray(h >= 0).astype(int)`.

Celda "Perceptrón simple"

Ya está hecha: une tus dos funciones. Ejecútala.

Celda "Definamos unos pesos y sesgo"

Usa entradas [0.7, −0.3], pesos [0.1, 0.8] y sesgo −0.1.

✅ **Comprobación:** imprime `Output:` y luego **0**. La cuenta es h = 0.07 − 0.24 − 0.1 = −0.27, que es menor que 0.

Celda `TODO (1 punto)`: pesos para OR

Busca por prueba y error pesos y sesgo que den **OR**: 0 en (0, 0) y 1 en los otros tres casos. Pruébalos con las 4 entradas usando tu función `perceptron`.

✅ **Comprobación:** una solución válida (pesos [1, 1], sesgo −0.5) imprime:

```
(0, 0) -> 0
(0, 1) -> 1
(1, 0) -> 1
(1, 1) -> 1
```

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported operand ... NoneType`	`combinacion_lineal` todavía devuelve `None`	Completa la función y vuelve a ejecutar esa celda
La salida es `True` / `False` en vez de 1/0	No convertiste a entero	Usa `.astype(int)`
Da 1 en lugar de 0 en el ejemplo	Olvidaste sumar el sesgo, o usaste `>` al revés	Revisa `+ b` y la condición `>= 0`

Guía del notebook `1_03_2_regla_de_aprendizaje`: el perceptrón aprende

Puntos: 4 · **Tiempo:** 25–35 min · **Clase:** 2 · **Archivo:** `notebooks/1_03_2_regla_de_aprendizaje.ipynb`
Teoría completa: Guía de la Clase 2, sección B4.

¿Qué vas a aprender?

- Que el perceptrón puede **aprender solo** sus pesos a partir de ejemplos.
- A medir qué tanto se equivoca (la **pérdida**) y a ver cómo baja con el entrenamiento.
- A dibujar la **frontera de decisión** (la recta que separa las clases).

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_03_2_regla_de_aprendizaje.ipynb` → kernel **nni**.
   Usa la carpeta del repo, porque el notebook importa el paquete `nni` que está ahí.
3. Ejecuta las celdas en orden con **Shift + Enter**.

> **En Colab:** primero ejecuta en una celda nueva `!git clone https://github.com/datahuba/curso-redes-neuronales` y `%cd curso-redes-neuronales/notebooks`.

La idea en 2 minutos

Regla de Rosenblatt, aplicada a cada ejemplo:

$W \leftarrow W + \eta \cdot (y - \hat{y}) \cdot X$ y $b \leftarrow b + \eta \cdot (y - \hat{y})$

- **error = $y - \hat{y}$**: 0 si acertó, +1 o −1 si se equivocó.
- **η** (tasa de aprendizaje): el tamaño de la corrección.
- **Época:** una pasada por todos los ejemplos.
- **SAE** (suma de errores absolutos): cuántos ejemplos se equivoca en total. Con etiquetas 0/1, es exactamente el número de errores.

Paso a paso

Celdas 1–2: datos sintéticos

Cargan las librerías y generan **400 puntos** de dos clases (rojos = 0, azules = 1), que se dibujan.

Celdas 3–6: la clase `Perceptron`

Ya está hecha. Empieza con pesos en cero. Ejecútalas.

Celda `TODO`: inicialización (1 punto)

Asigna pesos aleatorios a `clasificador.W` (dos números) y un sesgo a `clasificador.b`.

- Pista: `np.random.normal(loc=0, scale=1, size=(2,))` genera dos números al azar alrededor de 0.
- El sesgo puede empezar en 0.0.
- Para obtener siempre los mismos números (y poder compararlos), escribe antes `np.random.seed(0)`.

✅ **Comprobación:** con `np.random.seed(0)` los pesos son ≈ **[1.764, 0.400]**.

Celda `TODO`: `perdidaSAE` (1 punto)

Suma de los valores absolutos de (y_true − y_pred).

- Pistas: `np.abs` y `np.sum`.

✅ **Comprobación:** con la semilla 0, la pérdida inicial es **93** (se equivoca en 93 de 400 puntos). Con otra semilla sale otro número; lo importante es que sea un entero entre 0 y 400.

Celda: gráfica del clasificador sin entrenar

Dibuja la frontera con pesos al azar. Normalmente separa mal los colores.

> ⚠️ Con scikit-learn 1.9 o más nuevo, esta gráfica exige el argumento `n_classes=2`. La copia del repositorio ya lo trae. Si usas una versión vieja y sale `unexpected keyword argument 'n_classes'`, bórralo.

Celda: `ReglaDeAprendizaje` (1 punto)

Completa las dos líneas `None` con la regla de arriba:

- los pesos nuevos son los pesos actuales más tasa · error · `xi`;
- el sesgo nuevo es el sesgo actual más tasa · error.

Celdas de entrenamiento

`n_epocas = 3` suele quedarse corto. La gráfica muestra la pérdida por época.

Celda `TODO (2 puntos)`: ajustar hiperparámetros

Sube `n_epocas` (por ejemplo a 20–50) y prueba `tasa_aprendizaje` entre 0.01 y 0.5. Después vuelve a ejecutar las celdas de entrenamiento y de la gráfica final.

✅ **Comprobación** (semilla 0, tasa 0.1, 20 épocas):

- la pérdida baja de **93** a unos **37**;
- tras la última actualización, el notebook queda en **34 errores de 400**;
- la frontera separa la mayoría de los puntos rojos y azules.

No tiene que ser perfecto: los datos se mezclan un poco y una recta no puede separarlos al 100 %.

Para pensar

- ¿Por qué la pérdida a veces **sube** de una época a otra? → La regla corrige ejemplo por ejemplo; arreglar uno puede descomponer otro.
- ¿Llegará alguna vez a 0? → Solo si los datos son linealmente separables.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... missing ... 'n_classes'`	scikit-learn nuevo	Agrega `n_classes=2,` dentro de `DecisionBoundaryDisplay(...)`
`ModuleNotFoundError: No module named 'nni'`	Notebook abierto fuera del repo	Abre la carpeta `curso-redes-neuronales` completa
La pérdida no cambia	`perceptron.W` sigue en `None`, o no ejecutaste la celda de la regla	Completa la regla y ejecútala de nuevo
Resultados distintos cada vez	Pesos iniciales al azar	Usa `np.random.seed(0)` antes de inicializar

Guía del notebook `1_04_red_neuronal_simple`: una neurona con sigmoide

Puntos: 3 · **Tiempo:** 20 min · **Clase:** 3 · **Archivo:** `notebooks/1_04_red_neuronal_simple.ipynb`
Teoría completa: Guía de la Clase 3, secciones B1 a B4.

¿Qué vas a aprender?

- A programar la función **sigmoide**, un "escalón suave".
- A escribir una neurona como una **clase** de Python, con sus métodos.
- A comparar la sigmoide con la **tangente hiperbólica (tanh)**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_04_red_neuronal_simple.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- La neurona moderna hace lo mismo que el perceptrón: h = W·X + b y luego ŷ = f(h). Pero **f es suave**.
- **Sigmoide:** $\sigma(h) = 1 / (1 + e^{-h})$. Devuelve un número entre 0 y 1.
- **tanh:** $(e^{h} - e^{-h}) / (e^{h} + e^{-h})$. Devuelve un número entre −1 y 1.
- En NumPy, e^{x} se escribe `np.exp(x)`, y el producto punto W·X se escribe `np.dot(W, X)`.

Paso a paso

Celda "Cargamos paquetes"

Ejecútala.

Celda `TODO (1 punto)`: `sigmoide(h)`

Escribe la fórmula y devuelve el resultado.
⚠️ Ojo con el **signo menos** del exponente: es `np.exp(-h)`.

Celda `TODO (1 punto)`: clase `Neurona`

Completa los dos métodos:

- **`combinacion_lineal(self, X)`:** devuelve h = W·X + b. Usa `np.dot(self.W, X) + self.b`. Dentro de la clase, los pesos se llaman `self.W` y el sesgo `self.b`.
- **`forward(self, X)`:** calcula h llamando a `self.combinacion_lineal(X)` y devuelve `self.activacion(h)`.

Celda "Definamos unos pesos y sesgo"

Usa entradas [0.7, −0.3], pesos [0.1, 0.8] y sesgo −0.1.

✅ **Comprobación:** debe imprimir `Output: 0.4329`.

- La cuenta: h = 0.07 − 0.24 − 0.1 = −0.27, y σ(−0.27) = 1 / (1 + $e^{0.27}$) = 1 / 2.3100 ≈ 0.4329.
- Si te da **0.5671**, te falta el signo menos en `np.exp(-h)`.

Celda `TODO (1 punto)`: tanh y comparación

1. Escribe una función `tanh(h)` con la fórmula de arriba.
2. Crea otra neurona con los mismos pesos, sesgo y entradas, pero con `tanh` como activación.
3. Imprime las dos salidas.

✅ **Comprobación:**

- con tanh sale **−0.2636**, y con la sigmoide **0.4329**;
- puedes comparar tu tanh con `np.tanh(-0.27)`, que también da −0.2636.

Para comentar: tanh es negativa porque h < 0; la sigmoide nunca es negativa (vale entre 0 y 1).

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported format string passed to NoneType`	Una función o método devuelve `None`	Completa el `TODO` y ejecuta de nuevo esa celda **y** la de prueba
Da 0.5671	Falta el signo: `np.exp(h)`	Debe ser `np.exp(-h)`
`NameError: name 'W' is not defined`	Dentro de la clase escribiste `W`	Usa `self.W` y `self.b`
`ValueError: shapes not aligned`	Pesos y entradas de distinto tamaño	Ambos deben tener 2 elementos

Guía del notebook `1_05_descenso_por_gradiente_1D`: bajar por una parábola

Puntos: 2 · **Tiempo:** 15–20 min · **Clase:** 3 · **Archivo:** `notebooks/1_05_descenso_por_gradiente_1D.ipynb`
Teoría completa: Guía de la Clase 3, sección B5.

¿Qué vas a aprender?

- A programar la regla del **descenso por gradiente**: $x \leftarrow x - \eta \cdot f'(x)$.
- A detener un ciclo cuando el resultado ya casi no cambia (**criterio de parada**).

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_05_descenso_por_gradiente_1D.ipynb` → kernel **nni**.
3. El notebook guarda una gráfica en `notebooks/output/`. Esa carpeta ya viene en el repositorio; si no la tienes, haz `git pull` o créala.

La idea en 2 minutos

- Queremos el x que hace **mínima** a $f(x) = x^2$. La respuesta es $x = 0$.
- La **derivada** $f'(x) = 2x$ es la **pendiente**: indica hacia dónde sube la curva.
- Para bajar, damos un paso **en contra** de la pendiente: **$x \leftarrow x - \eta \cdot f'(x)$**.
- **η** (tasa de aprendizaje) es el tamaño del paso.

Paso a paso

Celda: `f` y `df`

Define la función x^2 y su derivada $2x$. Ejecútala.

Celda `TODO`: `gradient_descent_update(theta, learning_rate)`

Devuelve el valor nuevo de theta con la regla de arriba. La derivada ya existe: es `df(theta)`.

Celda "Prueba del método"

Empieza en $x = 5$ con tasa 0.1 y hace 5 pasos.

✅ **Comprobación:** `Historial de valores de x: [5, 4.0, 3.2, 2.56, 2.048, 1.6384]`. Puede mostrar más decimales, como 1.6384000000000003.

Cada paso multiplica x por 0.8, porque $x - 0.1 \cdot 2x = 0.8x$. Por eso se acerca a 0 cada vez más despacio.

Celda de la gráfica

Dibuja la parábola y los puntos rojos bajando hacia el fondo. También la guarda en `output/gradient_descent_1D.png`.

Celda `TODO (1 punto)`: detenerse con un umbral

Reescribe el ciclo con `while` para que se detenga cuando "el error" sea menor que 0.001, y cuenta las iteraciones.

Pistas:

1. Empieza con `x = 5` y un contador `iteraciones = 0`.
2. Dentro del `while`: calcula el x nuevo, suma 1 al contador, y decide si ya paras.
3. Elige y escribe tu **criterio de parada**.

✅ **Comprobación** (ambos criterios son válidos si los explicas):

Criterio	Iteraciones
El **cambio** $\|x_{nuevo} - x\| < 0.001$	**32** (x final ≈ 0.00396)
El **valor** $f(x) = x^2 < 0.001$	**23**

Para pensar

- ¿Qué pasa con tasa = 1.1? → Cada paso multiplica x por −1.2: oscila y **diverge**.
- ¿Y con 0.5? → Llega a 0 en un solo paso. Pasa solo en esta función tan simple.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: unsupported operand ... NoneType`	`gradient_descent_update` devuelve `None`	Completa la función
`FileNotFoundError: output/gradient_descent_1D.png`	No existe la carpeta `output`	`git pull`, o crea la carpeta `notebooks/output`
El historial **crece** en lugar de bajar	Sumaste en vez de restar	Es `theta - learning_rate * df(theta)`
El `while` nunca termina	La condición nunca se cumple, o la tasa es muy grande	Revisa el criterio; detén la celda con el botón ■

Guía del notebook `1_06_descenso_unpaso`: un paso de la regla delta

Puntos: 3 · **Tiempo:** 20 min · **Clase:** 4 · **Archivo:** `notebooks/1_06_descenso_unpaso.ipynb`
Teoría completa: Guía de la Clase 4, sección B2.

¿Qué vas a aprender?

- Los tres pasos con los que una neurona corrige sus pesos: **término de error**, **incremento** y **actualización**.
- A comprobar que, después de un paso, la neurona se equivoca **menos**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_06_descenso_unpaso.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos (regla delta)

Paso	Fórmula	En palabras
1. Término de error	$\delta = (y - \hat{y}) \cdot f'(h)$	Cuánto y hacia dónde corregir
2. Incremento	$\Delta w = \eta \cdot \delta \cdot x$ ($\Delta b = \eta \cdot \delta$)	Cuánto mover cada peso
3. Actualización	$w \leftarrow w + \Delta w$, $b \leftarrow b + \Delta b$	Mover los pesos

Con la sigmoide, $f'(h) = \sigma(h) \cdot (1 - \sigma(h))$. El notebook ya la trae como `sigmoid_prime`.

Paso a paso

Celda: `sigmoid`, `sigmoid_prime` y la clase `Neurona`

Ya están hechas. Ejecútala.

Celda `TODO (1 punto)`: `error_term(y, x, neurona)`

Pistas:

- h se obtiene con `neurona.combinacion_lineal(x)`;
- ŷ, con `neurona.forward(x)`;
- f′(h), con `sigmoid_prime(h)`;
- multiplica $(y - \hat{y})$ por f′(h).

Celda `TODO (1 punto)`: `incremento(eta, error_term, X)`

Devuelve η·δ·X.

Celda `TODO (1 punto)`: `actualizacion(neurona, incremento_W, incremento_b)`

Suma el incremento a `neurona.W`, y el del sesgo a `neurona.b`. Devuelve la neurona.

Celda "valores de ejemplo"

Usa tasa 2.0, x = [1, 1], y = 1, w = [0.1, 0.2] y b = 0.

Celda "Calcular la salida de la red"

Cambia `salida = None` por la predicción de la neurona, `neurona.forward(x)`. Lo demás ya llama a tus funciones.

✅ **Comprobación:**

Línea	Debe imprimir
Salida	≈ 0.5744
Término de error	≈ 0.1040
Incremento	≈ [0.2081 0.2081]
Incremento del sesgo	≈ 0.2081
Pesos actualizados	≈ [0.3081 0.4081]

Último punto (1 punto): ¿bajó la pérdida?

Agrega una celda que vuelva a predecir con la neurona actualizada y compare la pérdida ½(y − ŷ)² antes y después.

✅ **Comprobación:** la nueva salida es ≈ **0.7159**, más cerca de y = 1. La pérdida baja de **0.0905** a **0.0404**.

Errores frecuentes

Síntoma	Causa	Solución
Término de error ≈ −0.104	Escribiste (ŷ − y) en lugar de (y − ŷ)	El orden es **y − ŷ**
Incremento del sesgo con dos números	Pasaste `x` en lugar de `1`	Para el sesgo, la "entrada" es 1 (la celda ya lo hace)
`TypeError: ... NoneType`	Alguna función devuelve `None`	Completa las tres funciones y vuelve a ejecutarlas
Los pesos no cambian	`actualizacion` no guarda en `neurona.W`	Asigna `neurona.W = neurona.W + incremento_W`

Guía del notebook `1_07_descenso_por_gradiente`: entrenar una neurona completa

Puntos: 3 · **Tiempo:** 25–30 min · **Clase:** 4 · **Archivo:** `notebooks/1_07_descenso_por_gradiente.ipynb`
Teoría completa: Guía de la Clase 4, sección B3.

¿Qué vas a aprender?

- A entrenar una neurona con **todos** los ejemplos, durante muchas **épocas**.
- A inicializar los pesos con **Xavier**.
- A medir la **exactitud** con datos que la red no vio al entrenar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_07_descenso_por_gradiente.ipynb` → kernel **nni**.
   Usa la carpeta del repo: el notebook carga `data.csv`, `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Datos:** 400 alumnos que pidieron entrar a un posgrado.
  - Entradas: puntaje GRE, promedio GPA y el prestigio de su universidad (`rank`, en 4 columnas de 0/1). Son **6 entradas**.
  - Salida: `admit`, que vale 1 si fue admitido.
  - `data_prep.py` ya los escala y separa **360 para entrenar** y **40 para probar**.
- **Algoritmo por época:**
  1. Para cada alumno: predicción $\hat{y}$, término de error $\delta = (y - \hat{y})\cdot\hat{y}(1 - \hat{y})$, y **acumular** $\delta \cdot x$.
  2. Al final de la época: pesos += η · acumulado / número de alumnos.
- **Xavier:** pesos al azar entre $-1/\sqrt{n}$ y $+1/\sqrt{n}$ ($n = 6$ → entre −0.408 y +0.408).
- **Exactitud** = aciertos / predicciones. La red dice "admitido" si $\hat{y} > 0.5$.

Paso a paso

Celda 1: paquetes y datos

Carga todo y fija la semilla 42. Si da error, revisa que estés en la carpeta del repositorio.

Celda `TODO (1 punto)`: inicializar con Xavier

Pista: `np.random.uniform(-1/np.sqrt(n_features), 1/np.sqrt(n_features), size=n_features)`.

Celda: exactitud antes de entrenar

✅ **Comprobación:** con la semilla 42 sale **0.650**. Es casi lo mismo que adivinar "no admitido" siempre.

Celda: hiperparámetros

`epochs = 10` y `learnrate = 0.5`. Más adelante subirás las épocas.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los cuatro huecos siguiendo el algoritmo:

1. `output` → `nodo.forward(x)`.
2. `error_term` → (y − output) · output · (1 − output).
3. `incremento_w +=` → error_term · x.
4. Después del `for` de ejemplos: `weights +=` tasa · incremento acumulado, **dividido entre** `n_records`.

⚠️ Escribe `weights += ...` y no `weights = weights + ...`. La neurona `nodo` comparte el arreglo `weights`: con `+=` lo modificas "en el lugar" y la neurona ve el cambio. Con `=` creas un arreglo nuevo, la neurona se queda con los pesos viejos y **la pérdida no baja**.

Celda: gráfica de la pérdida

Debe bajar época tras época.

Celda: exactitud final (1 punto si supera 0.74)

✅ **Comprobación** (semilla 42, `learnrate = 0.5`):

Épocas	Pérdida al final	Exactitud
10	≈ 0.225	≈ 0.60
100	≈ **0.2055**	**0.775** ✓
1000	≈ 0.197	≈ 0.725

Para pensar

¿Por qué con 1000 épocas la pérdida baja pero la exactitud empeora? El conjunto de prueba es pequeño (40 alumnos) y la red se ajusta demasiado a los datos de entrenamiento: es **sobreajuste**.

Errores frecuentes

Síntoma	Causa	Solución
La pérdida no baja nada	`weights = weights + ...`	Usa `weights += ...`
`ModuleNotFoundError: data_prep` o `nni`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
`ValueError: shapes not aligned`	`size` distinto de `n_features`	`size=n_features`
`WARNING - La pérdida ha aumentado`	Tasa muy grande	Baja `learnrate` a 0.1

Síntoma	Causa	Solución
Exactitud distinta a la tabla	No ejecutaste la celda con `np.random.seed(42)`	Ejecuta todo desde la primera celda

Guía del notebook `1_08_adaline`: regresión con una neurona

Puntos: demostración, sin `TODO` · **Tiempo:** 10–15 min · **Clase:** 4 · **Archivo:** `notebooks/1_08_adaline.ipynb`
Teoría completa: Guía de la Clase 4, sección B4.

¿Qué vas a aprender?

- Que una neurona con **activación lineal** (ADALINE) sirve para **predecir números** (regresión).
- A ver el error como una **superficie** (un valle en 3D) que el descenso por gradiente tiene que bajar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_08_adaline.ipynb` → kernel **nni**. El notebook lee `files/Salary_dataset.csv`, así que hay que abrirlo desde la carpeta del repo.
3. Este notebook **no tiene** `TODO`: ejecútalo completo y lee cada resultado.

La idea en 2 minutos

- **ADALINE** (Widrow y Hoff, 1960): $\hat{y} = w \cdot x + b$, sin función escalón ni sigmoide. Es una **recta**.
- **Error:** el error cuadrático medio (MSE) entre los salarios reales y los predichos.
- **Regla** (descenso por gradiente con $f' = 1$): $w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$ y $b \leftarrow b + \eta \cdot (y - \hat{y})$.

Recorrido por las celdas

1. **Datos:** 30 personas, con sus años de experiencia y su salario. La gráfica muestra que a más experiencia, más salario.
2. **Clase `ADALINE`:** pesos al azar entre 0 y 1; `predict` calcula $w \cdot x + b$.
3. **Recta sin entrenar:** casi plana y pegada a 0, muy lejos de los puntos (los salarios valen decenas de miles).
4. **Superficie del error:** para muchas combinaciones de peso y sesgo calcula el MSE y lo dibuja en 3D. El punto más bajo de ese valle son los mejores parámetros.
5. **Entrenamiento:** 1000 pasadas con tasa 0.0001, ejemplo por ejemplo.
6. **Historial del MSE:** baja rápido y luego se aplana.
7. **Recta final:** pasa por en medio de los puntos.

✅ **Qué esperar:**

- El resultado cambia un poco cada vez, porque los pesos iniciales son al azar (no hay semilla). En una ejecución salió $w \approx 8\,400$ y $b \approx 32\,000$.
- La recta **óptima** (mínimos cuadrados) es **salario ≈ 9 450 · años + 24 848**. `docs/practica_04.py` llega exactamente a ella.

Para pensar

¿Por qué el notebook no llega exactamente a la recta óptima?

- Los datos no están **escalados**: los años van de 1 a 10.6 y los salarios de 37 000 a 122 000.
- Por eso hay que usar una tasa diminuta (0.0001), y con ella el **sesgo** aprende muy lento.
- Si se escalan los datos (restar la media y dividir entre la desviación estándar), el descenso llega al óptimo rápido. Eso hace `docs/practica_04.py`.

Pruébalo:

```
python docs/practica_04.py --graficar
```

Errores frecuentes

Síntoma	Causa	Solución
`FileNotFoundError: files/Salary_dataset.csv`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
La gráfica 3D no aparece	Falta matplotlib, o la celda dio error	`pip install matplotlib` con `(nni)` activo
Recta distinta a la de un compañero	Pesos iniciales al azar	Es normal; agrega `np.random.seed(0)` al principio para repetir resultados

Guía del notebook `1_09_red_multicapa_ejercicio`: una red con capa oculta

Puntos: 3 · **Tiempo:** 20–25 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_09_red_multicapa_ejercicio.ipynb`
Lectura: libro, Redes neuronales multicapa.

¿Qué vas a aprender?

- Qué es una **capa oculta** y por qué hace falta.
- A representar los pesos de cada capa como una **matriz**.
- A calcular la salida de una red de dos capas (**pase hacia adelante**) con productos de matrices.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_09_red_multicapa_ejercicio.ipynb` → kernel **nni**.
   El notebook usa el paquete `nni` y la imagen `files/test1.png` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

¿Por qué capas ocultas?

Una sola neurona solo traza **una recta**, y por eso no puede con XOR (Clases 2 y 4). Si ponemos una **capa de neuronas en medio** (la capa oculta), cada una traza su recta y la neurona de salida las combina. Así la red puede separar regiones con formas más complejas. Esta red también se llama **perceptrón multicapa** (MLP).

La red de este notebook

4 entradas → 3 neuronas ocultas → 2 salidas, como en la figura del notebook.

Pesos como matrices

- Entre la entrada (4) y la oculta (3) hay 4 × 3 = 12 pesos. Van en una matriz **W_1 de 4 filas y 3 columnas**: las filas son las entradas y las columnas, las neuronas de la capa siguiente.
- Entre la oculta (3) y la salida (2) hay una matriz **W_2 de 3 × 2**.

Pase hacia adelante (dos veces lo mismo de la Clase 4)

Paso	Fórmula	Tamaño del resultado
Capa oculta	$H_1 = X \cdot W_1 \ (+ B_1)$ y $A_1 = \sigma(H_1)$	1 × 3
Salida	$H_2 = A_1 \cdot W_2 \ (+ B_2)$ y $\hat{Y} = \sigma(H_2)$	1 × 2

La salida de la capa oculta, $\mathbf{A_1}$, es la entrada de la siguiente.

Regla de los tamaños: para multiplicar (a × b)·(b × c), el número del medio debe coincidir, y el resultado es (a × c). Por ejemplo, (1 × 4)·(4 × 3) = (1 × 3).

Paso a paso

Celda 1: paquetes

Importa NumPy y `nni`, y fija la semilla 42.

Celda: tamaños de las capas

Completa `N_input`, `N_hidden` y `N_output` según la figura: **4, 3 y 2**.

Celda `TODO (1 punto)`: pesos y entrada

1. **Media y desviación:** media 0 y una desviación pequeña. Una buena elección es **1/√(número de entradas)**, la inicialización normal escalada que vimos con Xavier.
2. **Matrices:** `np.random.normal(mean, stdev, size=(N_input, N_hidden))` para W_1, y de tamaño `(N_hidden, N_output)` para W_2. Usa las variables de tamaño, no números fijos.
3. **Entrada:** un vector aleatorio de tamaño `(1, N_input)`, por ejemplo con `np.random.normal(size=(1, N_input))`.

✅ **Comprobación:** imprime las formas. Deben ser W_1 **(4, 3)**, W_2 **(3, 2)** y X **(1, 4)**.

Celda `TODO (1 punto)`: calcular la salida

- `H_1` = producto de X por W_1 (`np.dot`).
- `H_2` = producto de A_1 por W_2. **Ojo:** se usa A_1 (después de la sigmoide), no H_1.

✅ **Comprobación:**

- la salida de la capa oculta tiene **3 números** entre 0 y 1;
- la salida de la red tiene **2 números** entre 0 y 1.

Con la semilla 42, y generando W_1, W_2 y X en ese orden con desviación 1/√4, sale:

- capa oculta ≈ [0.4489, 0.6991, 0.3965];
- red ≈ [0.3210, 0.3627].

Si generas en otro orden, los números cambian, pero los tamaños no.

Último punto (1 punto): sesgos

Agrega una celda nueva con un vector de sesgos para cada capa: **B_1 de tamaño (1, 3)** y **B_2 de tamaño (1, 2)**. Súmalos antes de cada sigmoide.

✅ **Comprobación:**

- con sesgos en cero, la salida es igual a la de antes;
- con otros valores, cambia;
- en total la red tiene 4·3 + 3 + 3·2 + 2 = **23 parámetros**.

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: shapes (1,4) and (3,2) not aligned`	Multiplicaste X por W_2, o H_1 en lugar de A_1	Sigue el orden: X·W_1 → σ → A_1·W_2 → σ
`TypeError: ... NoneType`	Algún tamaño o matriz sigue en `None`	Completa todas las variables
`AttributeError: module 'nni' has no attribute 'sigmoid'`	Notebook abierto fuera del repo, o `nni` viejo	Abre la carpeta `curso-redes-neuronales` y haz `git pull`
Salidas iguales a 0.5	Pesos o entradas todos en cero	Usa `np.random.normal`

Guía del notebook `1_10_retropropagacion_base`: retropropagación, un paso

Puntos: 3 · **Tiempo:** 25–30 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_10_retropropagacion_base.ipynb`
Lectura: libro, Retropropagación.

¿Qué vas a aprender?

- Cómo se **reparte la culpa del error** entre las capas de una red: la **retropropagación** (*backpropagation*).
- A calcular el término de error de la capa de salida y el de la capa oculta.
- A calcular los incrementos de **todas** las matrices de pesos.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_10_retropropagacion_base.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

En la Clase 4 aprendimos a corregir los pesos de **una** neurona, porque conocíamos su error (y − ŷ). Pero en una red con capa oculta, **¿cuál es el "error" de una neurona oculta?** Nadie nos dice cuál debió ser su salida.

La solución: mandar el error hacia atrás

1. Calcula el error en la **salida**, como siempre.
2. **Repártelo** hacia la capa oculta: cada neurona oculta recibe una parte proporcional al **peso** que la conecta con la salida (si influyó mucho, carga más culpa).
3. Con esos términos de error, cada capa se actualiza con la misma regla delta.

Capa	Término de error	Incremento de sus pesos
Salida	$\delta_o = (y - \hat{y}) \cdot f'(h_o)$	$\Delta w_ho = \eta \cdot \delta_o \cdot a_h$ (a_h = salida de la oculta)
Oculta	$\delta_h = \delta_o \cdot w_ho \cdot f'(h_h)$	$\Delta w_ih = \eta \cdot \delta_h \cdot x_i$

Con la sigmoide, $f' = \text{salida} \cdot (1 - \text{salida})$.

La red de este notebook

3 entradas → 2 neuronas ocultas → 1 salida, con pesos ya dados.

Paso a paso

Celdas 1–3: funciones y red

Definen la sigmoide, su derivada y la red:

- x = [0.5, 0.1, −0.2], objetivo = 0.6, tasa 0.5;
- pesos de entrada a oculta (3 × 2) y de oculta a salida (2).

Celda: pase frontal (ya está hecha)

Calcula, en este orden:

Valor	Resultado
Entrada a la capa oculta	x·W_ih = [0.24, −0.46]
Salida de la capa oculta	σ(...) ≈ [0.5597, 0.3870]
Entrada a la salida	0.5597·0.1 + 0.3870·(−0.3) ≈ −0.0601
Salida	σ(−0.0601) ≈ 0.4850

Celda `TODO (1 punto)`: términos de error

- `error` = objetivo − salida.
- `del_err_output` = error · salida · (1 − salida).
- `del_err_hidden` = del_err_output · `weights_hidden_output` · salida_oculta · (1 − salida_oculta). Es un vector de 2 números, uno por neurona oculta.

✅ **Comprobación:**

Variable	Valor
error	≈ 0.1150
del_err_output	≈ 0.02873
del_err_hidden	≈ [0.000708, −0.002045]

Celda `TODO (1 punto)`: incrementos y actualización

1. **Tasa:** el notebook pide proponer otra. Para compararte con estos valores, usa primero **0.5**.
2. **Oculta → salida:** `delta_w_h_o` = tasa · del_err_output · salida_oculta.
3. **Entrada → oculta:** `delta_w_i_h` = tasa · del_err_hidden · x, ordenado como **matriz 3 × 2**.
   Pista: `x[:, None]` convierte x en una columna (3 × 1); multiplicada por un vector de 2 da una matriz 3 × 2.
4. **Actualizar:** suma cada incremento a su matriz de pesos.

✅ **Comprobación** (tasa 0.5):

- **Incremento oculta → salida** ≈ [0.00804, 0.00556].
- **Incremento entrada → oculta** ≈

```
[[ 0.000177 -0.000511]
 [ 0.000035 -0.000102]
 [-0.000071  0.000204]]
```

- **Nuevos pesos oculta → salida** ≈ [0.10804, −0.29444].
- Con otra tasa, todos los incrementos se escalan en la misma proporción. Por ejemplo, con 1.0 son el doble.

Pregunta (1 punto)

¿Son del mismo tamaño las matrices de incrementos que las de pesos? Sí: cada peso necesita su propio incremento.

- La de entrada → oculta es **3 × 2**.
- La de oculta → salida tiene **2** elementos.

Puedes comprobarlo con `.shape`.

Para pensar

¿Por qué los incrementos de la capa oculta son tan pequeños? El error llega multiplicado por pesos pequeños (0.1 y −0.3) y por derivadas de la sigmoide (≤ 0.25). En redes con muchas capas, esto se repite y el error casi desaparece: es el problema del **gradiente que se desvanece**.

Errores frecuentes

Síntoma	Causa	Solución
`del_err_hidden` es un solo número	Olvidaste multiplicar por el vector `weights_hidden_output`	Debe dar 2 valores
`ValueError: operands could not be broadcast`	Multiplicaste vectores 3 y 2 sin `x[:, None]`	Convierte x en columna
Valores con el signo contrario	Escribiste salida − objetivo	Es **objetivo − salida**
`TypeError: ... NoneType`	`learnrate` sigue en `None`	Asígnale un número

Guía del notebook `1_11_retropropagacion_completo`: entrenar una red multicapa

Puntos: 3 · **Tiempo:** 30–40 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_11_retropropagacion_completo.ipynb`
Requisito: haber hecho `1_07` (entrenar una neurona) y `1_10` (un paso de retropropagación).

¿Qué vas a aprender?

- A juntar todo: pase hacia adelante, retropropagación y descenso por gradiente, durante muchas épocas.
- A dibujar la **curva de aprendizaje** e interpretarla.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_11_retropropagacion_completo.ipynb` → kernel **nni**.
   El notebook usa `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Mismos datos que en `1_07`:** admisiones a un posgrado, 6 entradas, 360 alumnos para entrenar y 40 para probar.
- **Red:** 6 entradas → **2 neuronas ocultas** → 1 salida, todas con sigmoide.
- **Cada época:**
  1. Pone los incrementos acumulados en cero.
  2. Para cada alumno: pase hacia adelante, error, término de error de la salida y de la oculta (igual que en `1_10`), y **acumula** los incrementos.
  3. Al terminar los alumnos, **actualiza** las dos matrices de pesos.

Paso a paso

Celdas 1–2: datos

- Estadísticas de las entradas: GRE y GPA ya están escalados (media ≈ 0, desviación ≈ 1).
- Clases: **243 no admitidos y 117 admitidos**.

Celda 3: tamaños e hiperparámetros

- `n_hidden = 2`, `epochs = 900` y `learnrate = 0.001`.
- Pesos iniciales al azar, normales con desviación $1/\sqrt{6}$, y semilla 21.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los huecos con lo que ya hiciste en `1_10`:

Hueco	Qué escribir
`hidden_input`	x·W_ih (`np.dot`)
`hidden_output`	sigmoide de lo anterior (`functions.sigmoid`)
`output`	sigmoide de hidden_output·W_ho
`error`	y − output
`output_error`	error · output · (1 − output)
`hidden_error`	output_error · `weights_hidden_output` · hidden_output · (1 − hidden_output)
`del_w_hidden_output +=`	output_error · hidden_output
`del_w_input_hidden +=`	hidden_error · x en forma de matriz (`x[:, None]`, como en `1_10`)
Actualizar `weights_... +=`	tasa · incremento acumulado

Dos opciones válidas para la actualización final:

- **A, sin dividir:** pesos += `learnrate` · acumulado, con `learnrate = 0.001`, como viene la plantilla.
- **B, promediado:** pesos += `learnrate` · acumulado / `n_records`, igual que en `1_07`. Así la tasa **no** depende del número de alumnos, pero hay que subirla a ≈ **0.5**.

✅ **Comprobación** (semilla 21, 900 épocas):

Opción	Pérdida inicial → final	Exactitud de prueba
A: 0.001 sin dividir	0.2646 → 0.2013	**0.725**
B: 0.5 con / n_records	0.2641 → 0.1999	**0.750**
⚠️ 0.005 con / n_records	0.2659 → 0.2516	0.425 (casi no aprende)

La última fila es una tasa **demasiado pequeña** para la opción B: con ella la red casi no aprende.

⚠️ **Detalle del notebook original:** el bloque que imprime `Train loss` calcula la pérdida usando **solo el último alumno** (`x`), no todos. Para una curva correcta, usa `features.values` en lugar de `x` en esas dos líneas.

Celda `TODO (1 punto)`: curva de aprendizaje

1. Antes del ciclo, crea una lista vacía, por ejemplo `historial = []`.
2. Al final de **cada** época, calcula la pérdida con todos los alumnos y agrégala con `.append`.
3. Grafica con `plt.plot(historial)`, poniendo nombres a los ejes (época y pérdida).

✅ **Comprobación:** la curva **baja rápido al principio y luego se aplana**.

Preguntas (1 punto)

- **¿La pérdida de entrenamiento disminuye?** Sí, de ≈ 0.26 a ≈ 0.20.

- **¿Cuál es la mejor pérdida que alcanzaste?** Anota el valor, las épocas y la tasa que usaste. Por ejemplo: "0.1999 con 900 épocas y tasa 0.5 promediada".

Para pensar

- **¿Por qué la exactitud (0.75) es casi igual a la de una sola neurona en `1_07` (0.775)?** Estos datos son casi linealmente separables, así que una capa oculta no ayuda mucho. Las capas ocultas brillan con problemas **no lineales**, como XOR o imágenes.
- **Reto:** sube `n_hidden` a 4 u 8. ¿Mejora la exactitud o aparece sobreajuste?

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: operands could not be broadcast together with shapes (6,2) (6,)`	Sumaste `hidden_error * x` sin convertir x en columna	Usa `hidden_error * x[:, None]`
La pérdida no baja	Tasa muy pequeña para la opción B, o te faltó `+=`	Revisa la tabla de opciones
La pérdida impresa sube y baja raro	El cálculo original usa solo el último alumno	Usa `features.values`
`ModuleNotFoundError: data_prep`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
Tarda mucho	Son 900 épocas × 360 alumnos	Es normal: unos segundos. Ten paciencia o baja las épocas

Guía del notebook `21_opt_gradiente_2d`: descenso por gradiente en dos dimensiones

Puntos: experimentos, sin `TODO` · **Tiempo:** 20 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/21_opt_gradiente_2d.ipynb`
Requisito: `1_05` (descenso por gradiente en una variable).

¿Qué vas a aprender?

- A extender el descenso por gradiente a **dos parámetros** a la vez.
- A leer un **mapa de contorno**, que es como un mapa topográfico de la función.
- A ver con tus propios ojos el efecto de la tasa de aprendizaje.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/21_opt_gradiente_2d.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Función esférica:** $f(x, y) = x^2 + y^2$. Es un "tazón" cuyo fondo está en (0, 0).
- **Gradiente:** con dos variables hay una derivada por cada una (**derivadas parciales**):
  - $\partial f/\partial x = 2x$, que se calcula como si y fuera un número fijo;
  - $\partial f/\partial y = 2y$.
  - Juntas forman el **gradiente** $\nabla f = (2x, 2y)$: una flecha que apunta hacia donde la función sube más rápido.
- **Regla:** igual que en 1D, pero para cada parámetro: $x \leftarrow x - \eta \cdot \partial f/\partial x$ y $y \leftarrow y - \eta \cdot \partial f/\partial y$.
- **Contorno:** cada color es una "altura". El punto rojo es el mínimo; la línea blanca, el camino del descenso.

Recorrido por las celdas

1. **Paquetes y el problema:** `nni.SphereProblem()` trae la función, sus derivadas y el área a dibujar (de −2 a 2).
2. **Gráficas:** el mapa de contorno (izquierda) y la superficie 3D (derecha).
3. **Clase** `GradientDescentOptimizer` (léela con calma):
   - `next_params`: aplica la regla a cada parámetro;
   - `optimize`: repite hasta llegar al máximo de iteraciones o hasta que los parámetros casi no cambien (`tolerance`);
   - `history`: guarda cada paso para dibujarlo.
4. **Hiperparámetros:** inicio (−1, −1), tasa 0.01 y 300 iteraciones.

5. **Optimizar.**
   ✅ **Comprobación:** con los valores originales, `Parámetros optimizados` ≈ **[−0.00233, −0.00233]** y `Mejor valor encontrado` ≈ **$1.09 \cdot 10^{-5}$**. Usa las 300 iteraciones: con 0.01 avanza lento.
6. **Camino en el contorno:** una línea recta desde (−1, −1) hacia el centro.
7. **Animación:** guarda un video `gradient_descent.mp4`.

> ⚠️ El video **necesita ffmpeg**. Si sale `RuntimeError: Requested MovieWriter (ffmpeg) not available`, cambia el final de esa línea por `.save('gradient_descent.gif', writer='pillow')`. Así guarda un GIF sin instalar nada más.

Experimentos sugeridos

Cambia `learning_rate` en la celda de hiperparámetros y vuelve a ejecutar desde ahí. Resultados reales desde (−1, −1), con un máximo de 300 iteraciones:

Tasa	Iteraciones	Dónde termina	Qué pasa
0.01	300 (el máximo)	(−0.0023, −0.0023)	Lento: no termina de llegar
0.1	56	(0, 0)	Bien
0.5	2	(0, 0)	Llega de un salto (solo en esta función)
0.9	66	(0, 0)	Llega, pero **zigzagueando** de un lado al otro
1.0	300	(−1, −1)	**Rebota para siempre** entre (−1, −1) y (1, 1)
> 1.0	—	se aleja	**Diverge**

¿Por qué? Cada paso multiplica x por $(1 - 2\eta)$:

- con $\eta = 0.9$, ese factor es −0.8: cambia de signo, pero se achica;
- con $\eta = 1$, es −1: cambia de signo sin achicarse nunca;
- con $\eta > 1$, crece en cada paso.

Para pensar

- Cambia el punto inicial a (2, −1.5). ¿El camino sigue siendo una recta? (En la esfera, sí.)
- ¿Qué pasaría en una función con forma de valle alargado, en lugar de un tazón redondo? Lo verás en el notebook 22.

Errores frecuentes

Síntoma	Causa	Solución
`RuntimeError: ... MovieWriter (ffmpeg) not available`	No hay ffmpeg	Guarda como GIF con `writer='pillow'`, o salta esa celda
Aviso `Locator attempting to generate 1001 ticks`	`levels=1000` en el contorno	Solo es un aviso; puedes bajar a `levels=100`

Síntoma	Causa	Solución
`AttributeError: module 'nni' has no attribute 'SphereProblem'`	Notebook abierto fuera del repo, o `nni` viejo	Abre la carpeta `curso-redes-neuronales` y haz `git pull`
`OverflowError`	Tasa > 1: los números crecen sin límite	Usa una tasa menor que 1

Guía del notebook `22_opt_momentum`: el optimizador con momentum

Puntos: experimentos, sin `TODO` obligatorios · **Tiempo:** 20–25 min · **Módulo:** optimización (clase 6) ·
Archivo: `notebooks/22_opt_momentum.ipynb`
Requisito: `21_opt_gradiente_2d`.

¿Qué vas a aprender?

- Qué es el **momentum** y por qué ayuda en valles estrechos.
- A comparar dos optimizadores sobre una función difícil, la de **Beale**.
- Que el momentum permite usar tasas de aprendizaje con las que el descenso simple **se descontrola**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/22_opt_momentum.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

La función de Beale tiene un **valle largo, estrecho y curvo**. Con el descenso simple, si la tasa es grande el algoritmo **rebota** contra las paredes del valle y se puede salir disparado; si es pequeña, avanza muy lento.

Momentum = recordar los pasos anteriores

En lugar de seguir solo el gradiente de este momento, el optimizador lleva un **promedio** de los gradientes recientes:

$m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$ y parámetros $\leftarrow$ parámetros $- \eta \cdot m$

- **β** (beta, entre 0 y 1) es cuánta "memoria" tiene. Con $\beta = 0.9$, el paso de ahora se parece mucho a los anteriores.
- Los rebotes de un lado a otro **se cancelan** en el promedio, y la dirección constante, a lo largo del valle, **se mantiene**.

Analogía: una bola rodando por un valle no cambia de dirección bruscamente con cada piedrita; sigue su camino.

Función de Beale: su mínimo está en (3, 0.5) y vale 0. El notebook dibuja el **logaritmo** de la función para que se vea el valle.

Recorrido por las celdas

1. **Paquetes y problema:** `nni.ReducedBealeProblem()`, que dibuja x entre 0 y 4.5 e y entre −2 y 2.
2. **Gráficas:** contorno (con el valle curvo) y superficie 3D.
3. **Clase `MomentumOptimizer`:** hereda del descenso por gradiente y solo cambia `next_params` para usar la fórmula de arriba.
   El comentario `TODO: initialize dinamically` señala que `self.momentum = np.zeros(2)` supone 2 parámetros. Una mejora opcional: crear el vector del tamaño de `params` la primera vez que se llama a `next_params`.
4. **Hiperparámetros:** inicio (2.5, 1.0), tasa 0.01, 500 iteraciones y β = 0.9.
5. **Optimizar.**
   ✅ **Comprobación** (valores originales): termina ≈ **(2.922, 0.480)** con f ≈ **$1.07 \cdot 10^{-3}$**.
6. **Camino en el contorno** y **animación**, que necesita ffmpeg; ver el aviso de abajo.

Experimentos (resultados reales, desde (2.5, 1.0), 500 iteraciones)

Para comparar, el notebook trae comentada la línea del descenso simple: quita el `#` de esa línea y pon `#` en la del momentum.

Tasa	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	f ≈ $6.95 \cdot 10^{-4}$	f ≈ $7.37 \cdot 10^{-4}$	f ≈ $1.07 \cdot 10^{-3}$
0.03	**diverge** (`OverflowError`)	f ≈ $1.9 \cdot 10^{-6}$ ✓	**f ≈ $8.0 \cdot 10^{-7}$** ✓

Cómo leer la tabla:

- Con tasa pequeña (0.01) los tres llegan casi igual. El momentum no acelera aquí, porque esta versión **promedia**: no amplifica los pasos.
- Con tasa grande (0.03) el descenso simple rebota hasta "explotar", mientras que el momentum **suaviza los rebotes** y llega prácticamente al mínimo (3, 0.5).
- **Conclusión:** el momentum permite usar tasas más grandes con seguridad.

⚠️ **Crea un optimizador nuevo cada vez que pruebes otros valores.** El objeto guarda su `momentum` de la ejecución anterior.

Para pensar

- ¿Qué pasa con β = 0.99? → Tiene demasiada memoria: tarda en frenar y en girar con la curva del valle (con tasa 0.01 queda en f ≈ 0.12).
- ¿Y con β = 0? → La fórmula se vuelve el descenso simple.

Errores frecuentes

Síntoma	Causa	Solución
`OverflowError: (34, 'Result too large')`	El descenso simple divergió con tasa grande	Es justamente lo que se quiere mostrar; usa momentum o baja la tasa

Síntoma	Causa	Solución
`RuntimeError: ... ffmpeg not available`	La animación necesita ffmpeg	Cambia a `.save('momentum_beta.gif', writer='pillow')`
Resultados raros al repetir	Reusaste el mismo optimizador	Crea uno nuevo en cada prueba
`AttributeError: ... 'GradientDescentOptimizer'`	`nni` viejo o notebook fuera del repo	`git pull` y abre la carpeta del repo

Guía del notebook `23_opt_lr_decay`: decaimiento de la tasa de aprendizaje

Puntos: 3 · **Tiempo:** 25 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/23_opt_lr_decay.ipynb`
Requisito: `21` y `22`.

> En el repositorio original del curso este notebook está **vacío**. El de este repositorio es nuevo, hecho con el mismo estilo y la misma función de Beale que el 22.

¿Qué vas a aprender?

- A **reducir la tasa de aprendizaje** mientras se entrena: grande al principio para avanzar rápido, pequeña al final para afinar.
- Tres reglas de decaimiento y cómo programarlas.
- A crear tu propio optimizador **heredando** del de `nni`.

Antes de empezar

1. `conda activate nni`, y `git pull` para tener este notebook.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/23_opt_lr_decay.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

Con una tasa **fija** hay un dilema: si es grande, rebota cerca del mínimo; si es pequeña, tarda muchísimo. Con **decaimiento**, empezamos grande y la vamos bajando.

Regla	Fórmula	En Python
Escalonada	$\eta_0 \cdot \gamma^{(t // s)}$	cada `cada` iteraciones multiplica por `gamma`
Inversa	$\eta_0 / (1 + k \cdot t)$	baja rápido al inicio y luego despacio
Exponencial	$\eta_0 \cdot e^{(-k \cdot t)}$	baja suave y continuo (`math.exp`)

- t es el número de iteración (0, 1, 2…).
- `t // s` es la división entera: 250 // 100 = 2.

Paso a paso

Celdas 1–3: paquetes y función de Beale

Igual que en el notebook 22. El mínimo está en (3, 0.5).

Celda `TODO (1 punto)`: las tres reglas

Escribe las tres funciones con las fórmulas de la tabla.

✅ **Comprobación:** la celda siguiente debe imprimir:

```
t =   0:  escalonada = 0.05000   inversa = 0.05000   exponencial = 0.05000
t = 100:  escalonada = 0.02500   inversa = 0.02500   exponencial = 0.01839
t = 250:  escalonada = 0.01250   inversa = 0.01429   exponencial = 0.00410
```

Celda `TODO (1 punto)`: `DecayOptimizer.next_params`

Sigue los 4 pasos escritos en los comentarios:

1. Calcula la tasa de esta iteración: `self.regla(self.lr0, self.t)`.
2. Guárdala en `self.tasas` (con `.append`) y en `self.learning_rate`.
3. Suma 1 a `self.t`.
4. Devuelve los parámetros nuevos: para cada par (parámetro, gradiente), parámetro − tasa · gradiente. Pista: es igual a la clase del notebook 21, pero con la tasa que calculaste.

Celda: comparar tasa fija y decaimiento

Desde (1.0, 1.0), con $\eta_0 = 0.05$ y 500 iteraciones.

✅ **Comprobación:**

```
Tasa fija 0.05:       termina en (2.866, 0.409),  f = 5.76e-02
Con decaimiento:      termina en (2.980, 0.495),  f = 6.34e-05
```

Con decaimiento, el error es casi **mil veces menor**. La gráfica muestra:

- con tasa fija, el camino **rebotando** dentro del valle;
- con decaimiento, el camino asentándose en el mínimo;
- la tasa de aprendizaje bajando "en escalones".

Celda `TODO (1 punto)`: experimenta

✅ **Resultados de referencia** (desde (1.0, 1.0), 500 iteraciones):

Configuración	Termina en	f
Escalonada (γ = 0.5 cada 100)	(2.980, 0.495)	$6.3 \cdot 10^{-5}$
Inversa (k = 0.01)	(2.975, 0.494)	$1.0 \cdot 10^{-4}$
Exponencial (k = 0.01)	(2.908, 0.476)	$1.5 \cdot 10^{-3}$
Tasa fija 0.01	(2.910, 0.477)	$1.4 \cdot 10^{-3}$
Escalonada con γ = 0.8	**(2.998, 0.499)**	$\mathbf{7.8 \cdot 10^{-7}}$

La exponencial con k = 0.01 **baja demasiado rápido**: en la iteración 250 la tasa ya es 0.004, y se "congela" antes de llegar. Quedó casi igual que la tasa fija pequeña.

Para usar γ = 0.8 sin cambiar la función, puedes pasar una regla "envuelta": `regla=lambda lr0, t: tasa_escalonada(lr0, t, gamma=0.8)`.

Preguntas (orientación)

- **¿Por qué con 0.05 fijo no llega al mínimo?** El paso es demasiado grande para el valle estrecho: salta de una pared a la otra.
- **¿Qué ventaja tiene empezar grande y reducir?** Avanza rápido cuando está lejos y afina cuando está cerca.
- **¿Y si la tasa decae demasiado rápido?** Se queda "congelado" antes de llegar, como la exponencial de la tabla.

Errores frecuentes

Síntoma	Causa	Solución
`TypeError: ... NoneType`	Alguna regla o `next_params` devuelve `None`	Completa todos los `return`
La tasa nunca cambia	No incrementas `self.t`	Agrega `self.t += 1`
La gráfica de la tasa sale vacía	No guardas en `self.tasas`	`self.tasas.append(tasa)`
Resultados distintos al repetir	Reusaste el mismo optimizador (su `t` sigue contando)	Crea uno nuevo en cada prueba

Guía del notebook `3_1_capa_convolucional`: la operación de una capa convolucional

Puntos: 4 · **Tiempo:** 30 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_1_capa_convolucional.ipynb`

¿Qué vas a aprender?

- Qué es una **imagen** para la computadora: una matriz de números.
- Qué es un **kernel** (filtro) y cómo se **desliza** sobre la imagen: la correlación cruzada.
- A programar a mano la operación central de las **redes convolucionales** (CNN), que se usan para reconocer imágenes.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_1_capa_convolucional.ipynb` → kernel **nni**.
   El notebook abre `files/Lenna.png`, así que debe abrirse desde la carpeta del repo.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

La imagen

Una imagen en color son 3 matrices: rojo, verde y azul. Aquí se usa solo el **canal rojo**, una matriz de **256 × 256** con valores de 0 (negro) a 255 (blanco).

El kernel y la correlación cruzada

Un **kernel** es una matriz pequeña, aquí de 3 × 3. Se coloca sobre cada píxel de la imagen y se calcula:

$G[i, j] = \Sigma_u \Sigma_v H[u, v] \cdot I[i + u, j + v]$

En palabras:

1. Pon el kernel centrado en el píxel (i, j).
2. Multiplica cada número del kernel por el píxel que tiene debajo.
3. Suma los 9 productos.
4. Ese total es el píxel (i, j) de la imagen de salida G.

- Los índices u y v van de −k a k. Con un kernel de 3 × 3, k = 1, y van de −1 a 1.
- En la matriz de Python, el elemento H[u, v] de la fórmula está en `H[u + k, v + k]`, porque los índices de Python empiezan en 0.

El kernel de este notebook: detector de bordes verticales

```
[[-1/3  0  1/3]
 [-1/3  0  1/3]
 [-1/3  0  1/3]]
```

Resta la columna de la izquierda y suma la de la derecha. Por eso da un número **grande y positivo** donde la imagen pasa de **oscuro a claro** de izquierda a derecha (un borde vertical), y **negativo** en el caso contrario.

Bordes de la imagen

En las orillas el kernel "se sale" de la imagen. Con **padding 0** (sin relleno), esos píxeles no se calculan. La salida es más pequeña: 256 − 2k = **254 × 254**.

Paso a paso

Celdas 1–2: paquetes y la imagen

✅ **Comprobación:** `Dimensiones de la imagen: (256, 256)` y `Tipo de datos de la matriz: uint8`.

Celda del kernel

Construye el detector de bordes y lo muestra.

Celda `TODO`: las dos funciones (3 puntos)

`correlacionPixel(H, I, i, j)` (1 punto)

- Con dos `for` anidados: `u` y `v` en `range(-k, k + 1)`. Acumula en `sumatoria` el producto `H[u + k, v + k] * I[i + u, j + v]`.
- Al final, `resultado` debe ser un número de NumPy, porque la función hace `resultado.astype(int)`. Por ejemplo, `resultado = np.array(sumatoria)`.
- Atajo opcional: `np.sum(H * I[i-k:i+k+1, j-k:j+k+1])` hace lo mismo en una línea.

`correlacionCruzadaValida(H, I)`

- *(1 punto)* `G` debe ser una matriz de ceros **del mismo tamaño** que I: `np.zeros((m, n))`.
- *(1 punto)* Al final, quita las orillas que no se llenaron. Pista: recorta `k` filas y `k` columnas de cada lado, `G[k:m-k, k:n-k]`.

Celda "Prueba"

Aplica tu función, luego la ReLU (que deja en 0 los negativos), y dibuja tres imágenes.

✅ **Comprobación:**

- la imagen "Correlación cruzada" se ve gris, con los contornos marcados en claro y oscuro;
- "ReLU aplicada" se ve negra, con **solo algunos bordes verticales** en blanco;
- debe parecerse a la figura `files/ccnout.png` del notebook;
- el resultado mide **254 × 254**: compruébalo con `resultado.shape`.

Pregunta (1 punto)

¿Por qué tras la ReLU solo se ven algunos bordes verticales?

- El kernel da valores **positivos** en los bordes donde se pasa de **oscuro (izquierda) a claro (derecha)**, y **negativos** en los bordes de claro a oscuro.
- La ReLU convierte todos los negativos en 0, así que esos bordes desaparecen.
- Tampoco se ven los bordes **horizontales**, porque este kernel solo compara izquierda contra derecha.

Para pensar

- Si **volteas** el kernel (los +1/3 a la izquierda), ¿qué bordes aparecen? → Los otros, los que antes eran negativos.
- En una CNN, los números del kernel **no se eligen a mano**: la red los aprende. Eso es el notebook `3_2`.

Errores frecuentes

Síntoma	Causa	Solución
`AttributeError: 'float' object has no attribute 'astype'`	`resultado` es un número normal de Python	`resultado = np.array(sumatoria)`
`IndexError: index 256 is out of bounds`	Recorriste hasta el borde	Los `for` deben ir de `k` a `m-k` (ya vienen así)
La imagen sale con un marco negro	No recortaste G	`return G[k:m-k, k:n-k]`
Valores raros, enormes o negativos de golpe	Operaste con enteros `uint8`, que "dan la vuelta" al pasar de 255	Multiplica por el kernel, que es float, o convierte con `I.astype(float)`
`FileNotFoundError: files/Lenna.png`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
Tarda unos segundos	Son 254 × 254 píxeles × 9 productos con `for`	Es normal

Guía del notebook `3_2_entrenar_capa_conv`: reto, entrenar un kernel convolucional

Puntos: reto, abierto · **Tiempo:** 45–90 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_2_entrenar_capa_conv.ipynb`
Requisito: `3_1` (correlación cruzada) y `1_07` (descenso por gradiente).

¿Qué vas a aprender?

- Que los números de un kernel se pueden **aprender** con descenso por gradiente, igual que los pesos de una neurona.
- A calcular el gradiente de una capa convolucional.
- Trucos prácticos que hacen que un entrenamiento real funcione: **normalizar**, **centrar**, **vectorizar** y usar **momentum**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_2_entrenar_capa_conv.ipynb` → kernel **nni**.
   Usa `files/Lenna.png`, `files/sobel_horizontal.png` y el paquete `nni`.

El reto

Tenemos:

- la **entrada**: el canal rojo de la foto (256 × 256);
- el **objetivo**: la misma foto procesada con un detector de **bordes horizontales** (filtro de Sobel), también 256 × 256.

Hay que encontrar los **25 números de un kernel de 5 × 5** tales que ReLU(correlación(kernel, entrada)) ≈ objetivo.

Es el problema de la Clase 4, con dos cambios:

- los "pesos" son los 25 números del kernel;
- la "predicción" es una imagen entera.

La idea: el gradiente de un peso del kernel

Llamemos:

- G = correlación(K, I);
- P = ReLU(G), la predicción;
- T, el objetivo;
- ECM = promedio de $(P - T)^2$.

Un número del kernel, K[u, v], participa en **todos** los píxeles: en cada uno se multiplica por I[i + u, j + v]. Por la regla de la cadena:

∂ECM/∂K[u, v] = (2/N) · Σ sobre todos los píxeles (i, j) de: (P − T)[i, j] · [G[i, j] > 0] · I[i + u, j + v]

- (P − T) es el error en ese píxel.
- [G > 0] vale 1 si la ReLU dejó pasar el valor y 0 si no: la derivada de la ReLU.
- I[i + u, j + v] es el píxel que se multiplicó por ese peso.

Luego se aplica el descenso por gradiente: **K ← K − η·gradiente**.

Paso a paso

Celdas 1–3: imágenes

✅ **Comprobación:** ambas imágenes miden **(256, 256)**, son `uint8`, y el objetivo va de 0 a 255.

Celda: kernel aleatorio y pase frontal

El kernel es aleatorio y la celda no fija semilla, así que el resultado cambia en cada ejecución. Muy a menudo la predicción sale **toda en negro** (mínimo 0.0 y máximo 0.0): como todos los píxeles de la foto son positivos, si los 25 números del kernel suman un valor negativo, casi todos los G son negativos y la ReLU los deja en 0.

Así **ningún peso recibe gradiente** (es una "ReLU muerta"), y el descenso no puede empezar. Es la primera trampa del reto.

✅ **Comprobación:** cuando la predicción sale toda negra, el `Error cuadrático medio` es ≈ **1185.46**, que es exactamente el error de predecir todo en 0. Si te sale otro número, tu kernel aleatorio dio valores positivos: es normal.

Celdas con las funciones sugeridas

El notebook propone calcular el gradiente **píxel por píxel** con tres funciones auxiliares. Eso es correcto, pero son 252 × 252 píxeles × 25 pesos ≈ 1.6 millones de operaciones de Python **por época**: tardaría minutos por época.

Estrategia que funciona (probada)

1. **Normaliza:** divide la imagen y el objetivo entre 255, para que queden entre 0 y 1. Así los gradientes no son gigantes.
2. **Centra la entrada:** réstale su promedio. Así G ya no es "todo negativo" al inicio y la ReLU no muere. Como el kernel buscado suma 0, centrar no cambia la solución.
3. **Vectoriza:** en lugar de recorrer píxel por píxel, prepara las 25 versiones **desplazadas** de la imagen, una por cada (u, v). Por ejemplo, `I[k+u : m-k+u, k+v : n-k+v]`.
   - La correlación completa es la suma de cada versión desplazada por su peso.
   - El gradiente de cada peso es el promedio de (error · máscara · su versión desplazada) por 2.
   - Todo con operaciones de NumPy sobre matrices completas: cada época tarda milisegundos.
4. **Usa momentum** (notebook 22): V ← β·V + gradiente y K ← K − η·V, con β = 0.9. La imagen tiene píxeles vecinos muy parecidos, lo que hace el valle del error muy alargado: justo donde el momentum ayuda.
5. **Hiperparámetros** que funcionaron:
   - kernel inicial `np.random.normal(size=(5, 5))` con semilla 42;

- tasa η = 2.0;
- β = 0.9;
- **2000 épocas**, unos 5 segundos.

✅ **Resultado de referencia:**

- **ECM:** de ≈ 31 600 a ≈ **29**, en unidades de píxel².
- **Kernel aprendido:** prácticamente el **filtro de Sobel** (bordes horizontales), en el centro del 5 × 5:

```
[[ 0.05 -0.01 -0.09 -0.02  0.07]
 [-0.03 -0.86 -1.77 -0.89 -0.11]
 [-0.05 -0.03 -0.01  0.04  0.06]
 [ 0.05  0.93  1.79  0.84  0.06]
 [-0.05 -0.01  0.08  0.03 -0.08]]
```

  Compáralo con Sobel: fila de arriba −1, −2, −1; centro 0; fila de abajo +1, +2, +1.
- **Imagen:** la predicción es casi idéntica al objetivo.

Para pensar

- **¿Por qué la red "descubrió" el filtro de Sobel?** Porque el objetivo se hizo con él. Una CNN real aprende sus filtros del mismo modo, pero el objetivo es una tarea (reconocer gatos, por ejemplo) y no una imagen dada.
- **¿Por qué funciona mejor con momentum?** Sin él, con tasas grandes el entrenamiento rebota y "mata" la ReLU; con tasas pequeñas avanza muy lento.

Errores frecuentes

Síntoma	Causa	Solución
La predicción siempre es negra y el ECM no cambia	ReLU muerta: todos los G son negativos	Centra la entrada; inicializa con valores pequeños
Cada época tarda minutos	Cálculo píxel por píxel con `for`	Vectoriza con las 25 versiones desplazadas
El ECM sale `nan` o explota	Tasa demasiado grande	Baja η, o normaliza entre 0 y 1 si no lo hiciste
ECM gigantesco al inicio	Trabajas con valores 0–255	Divide imagen y objetivo entre 255
La forma no coincide con el objetivo	La correlación válida mide 252 × 252	Recorta el objetivo igual: `T[k:m-k, k:n-k]`